Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

PROGRAMAÇÃO I

ENGENHARIA INFORMÁTICA

Ficha 5 – Vetores e *Strings*

1. Pretende-se o desenvolver um programa para gerir as notas finais dos estudantes avaliados na UC de Programação.

   O programa deverá obter a quantidade de estudantes avaliados e as respetivas notas, calcular e apresentar a média das classificações, bem como as notas obtidas por cada estudante. Deverá também permitir a alteração de uma determinada nota.

   Para auxiliar na elaboração do programa deverá implementar, e ir testando, o definido nas seguintes alíneas:

   a) Defina a estrutura de dados necessária para armazenar as notas finais obtidas pelos alunos inscritos na UC de Programação, que tem o máximo de 25 inscritos.

   b) A função **`lerQuantidadeAvaliados()`** que solicite ao utilizador, e obtenha, a quantidade de estudantes avaliados.

   c) A função **`lerNotas()`** que solicite ao utilizador, e obtenha, as notas finais dos estudantes avaliados.

   d) A função **`mostrarDados()`** que apresente no monitor as notas dos estudantes avaliados.

   e) A função **`calculaMedia()`** que calcule, e devolva, a média das notas obtidas pelos estudantes.

   f) A função **`alteraNota()`** que permita alterar uma determinada nota.

   g) A função **`notasPositivas()`** que apresente no monitor as notas positivas, indicando a percentagem de estudantes que obtiveram notas positivas.

2. Pretende-se acrescentar as seguintes funcionalidades ao programa anterior:

   a) Definir estruturas de dados para armazenar os números e os nomes dos estudantes.

   b) Elaborar a função **`lerDadosEstudante()`** que obtenha a informação referente ao número (1-9999) e ao nome de um estudante avaliado.

   Alterar também as funções **`mostrarDados()`** e **`lerNotas()`** para incluir essa informação.

   c) Alterar a função **`alteraNota()`** para se solicitar a identificação do estudante a que se pretende alterar a nota.

3. Elabore um programa que auxilie na gestão da ocupação dos 10 quartos de uma determinada pensão. O menu principal deverá ser o seguinte:

```
(R)eservar
(D)esmarcar reserva / desocupar
(V)er estado de ocupações
(S)air
       OPCAO ->
```

(R)eservar

Esta opção apresenta o estado das ocupações e solicita ao utilizador o quarto a reservar.

```
 0 -> Ocupado
 1 -> Livre
 2 -> Livre
 3 -> Ocupado
 4 -> Livre
 5 -> Ocupado
 6 -> Livre
 7 -> Livre
 8 -> Livre
 9 -> Ocupado

Indique o nº do quarto que pretende reservar:
```

(D)esmarcar reserva / desocupar

Esta opção apresenta o estado das ocupações e solicitar ao utilizador o quarto a desmarcar.

(V)er estado de ocupações

Nesta opção deverá ser mostrado o estado de ocupações.

Na implementação do programa deverão ser elaboradas as seguintes funções:

a. **`menuPrincipal()`**: apresenta o menu principal, e obtém a seleção do utilizador.

b. **`inicializarQuartos()`**: coloca todos os quartos como livres. (sugestão: o vetor poderá ser de inteiros onde 0 corresponde a livre e 1 corresponde a ocupado).

c. **`estadoQuartos()`**: indica o estado (livre ou ocupado) de todos os quartos.

d. **`quartosNumEstado()`**: contabiliza o número de quartos que se encontram livres ou ocupados (conforme o estado que o utilizador pretender analisar).

e. **`leQuarto`**(): solicita ao utilizador o número do quarto que pretende reservar ou desmarcar. Deverá também validar o número do quarto e assegurar que o quarto se encontra livre se se pretender reservar ou ocupado se se pretender desmarcar.

4. Elabore um programa que solicite ao utilizador uma *string* e que contenha um conjunto de funções que permitam apresentar no monitor a seguinte informação.
   a) Quantidade de vogais da string.
   b) A *string* invertida.
   c) A *string* sem os espaços em branco, e a quantidade suprimida de espaços em branco.

5. Elabore um programa que obtenha um nome a partir do teclado e execute o seguinte conjunto de operações:
   a) Escreva no ecrã as suas iniciais;
   b) Indique quantas palavras existem no nome;
   c) Escreva o nome no formato: *Apelido, Nome*.

6. O clube de atletismo de Leiria pretende organizar um campeonato no qual participarão no máximo 20 atletas. O campeonato é composto por um conjunto de 6 corridas.

   Pretende-se que elabore um programa que auxilie o clube no registo dos tempos (em segundos) obtidos pelos atletas levou em cada uma das corridas, e na elaboração de algumas estatísticas.
   a) Declare as estruturas de dados para armazenar os números dos atletas, e os tempos obtidos pelos atletas em cada uma das corridas.
   b) Elabore a função **`ler_tempo_atletas_corrida()`**, que regista o tempo dos atletas, numa determinada corrida.
   c) Elabore a função **`mostrar()`**, que apresente o número do atleta e o tempo registado em cada uma das corridas em que participou.
   d) Elabore a função **`media_corrida()`**, que calcula e apresenta o tempo médio registado para os atletas que participaram numa determinada corrida.
   e) Elabore a função **`vencedor_corrida()`**, que apresente o número do atleta que venceu uma determinada corrida, e o respetivo tempo.
   f) Elabore a função **`vencedor_campeonato()`**, que apresente o número do atleta que venceu o campeonato.

      Considera-se como vencedor do campeonato o atleta que venceu o maior número de corridas. Como fator de desempate considera-se o menor tempo total utilizado para concluir todas as corridas do campeonato.

Anexo A – Tabela ASCII

Binário	Decimal	Hexa	Char	Binário	Decimal	Hexa	Char	Binário	Decimal	Hexa	Char
0010 0000	32	20		0100 0000	64	40	**@**	0110 0000	96	60	**`**
0010 0001	33	21	**!**	0100 0001	65	41	**A**	0110 0001	97	61	**a**
0010 0010	34	22	**"**	0100 0010	66	42	**B**	0110 0010	98	62	**b**
0010 0011	35	23	**#**	0100 0011	67	43	**C**	0110 0011	99	63	**c**
0010 0100	36	24	**$**	0100 0100	68	44	**D**	0110 0100	100	64	**d**
0010 0101	37	25	**%**	0100 0101	69	45	**E**	0110 0101	101	65	**e**
0010 0110	38	26	**&**	0100 0110	70	46	**F**	0110 0110	102	66	**f**
0010 0111	39	27	**'**	0100 0111	71	47	**G**	0110 0111	103	67	**g**
0010 1000	40	28	**(**	0100 1000	72	48	**H**	0110 1000	104	68	**h**
0010 1001	41	29	**)**	0100 1001	73	49	**I**	0110 1001	105	69	**i**
0010 1010	42	2A	*	0100 1010	74	4A	**J**	0110 1010	106	6A	**j**
0010 1011	43	2B	+	0100 1011	75	4B	**K**	0110 1011	107	6B	**k**
0010 1100	44	2C	**,**	0100 1100	76	4C	**L**	0110 1100	108	6C	**l**
0010 1101	45	2D	-	0100 1101	77	4D	**M**	0110 1101	109	6D	**m**
0010 1110	46	2E	**.**	0100 1110	78	4E	**N**	0110 1110	110	6E	**n**
0010 1111	47	2F	/	0100 1111	79	4F	**O**	0110 1111	111	6F	**o**
0011 0000	48	30	**0**	0101 0000	80	50	**P**	0111 0000	112	70	**p**
0011 0001	49	31	**1**	0101 0001	81	51	**Q**	0111 0001	113	71	**q**
0011 0010	50	32	**2**	0101 0010	82	52	**R**	0111 0010	114	72	**r**
0011 0011	51	33	**3**	0101 0011	83	53	**S**	0111 0011	115	73	**s**
0011 0100	52	34	**4**	0101 0100	84	54	**T**	0111 0100	116	74	**t**
0011 0101	53	35	**5**	0101 0101	85	55	**U**	0111 0101	117	75	**u**
0011 0110	54	36	**6**	0101 0110	86	56	**V**	0111 0110	118	76	**v**
0011 0111	55	37	**7**	0101 0111	87	57	**W**	0111 0111	119	77	**w**
0011 1000	56	38	**8**	0101 1000	88	58	**X**	0111 1000	120	78	**x**
0011 1001	57	39	**9**	0101 1001	89	59	**Y**	0111 1001	121	79	**y**
0011 1010	58	3A	**:**	0101 1010	90	5A	**Z**	0111 1010	122	7A	**z**
0011 1011	59	3B	**;**	0101 1011	91	5B	**[**	0111 1011	123	7B	**{**
0011 1100	60	3C	<	0101 1100	92	5C	\	0111 1100	124	7C	|
0011 1101	61	3D	=	0101 1101	93	5D	**]**	0111 1101	125	7D	**}**
0011 1110	62	3E	>	0101 1110	94	5E	^	0111 1110	126	7E	~
0011 1111	63	3F	**?**	0101 1111	95	5F	_				

Anexo B – Protótipos de funções úteis para caracteres e para Cadeias de caracteres ("*Strings*")

```
int isalpha(int c);
int isdigit(int c);
int islower(int c);
int isupper(int c);
int isspace(int c);
char tolower(int c);
char toupper(int c);

char *gets(char *str);
char *fgets(char *str, int LIM, FILE *stream);
int puts(char *str);
int strcmp(char s1[ ], char s2[ ]);
int strlen(char s[ ]);
void strcpy(char destino[], char origem[ ]);
void strcat (char destino[], char origem[ ]);
```

Exemplo:

```
strcpy(s1, "ola"); /* copia string "ola" vara vetor s1 */
```